

Migrations الدرس 5: قواعد البيانات والـ

جدول المحتويات

1. مقدمة في قواعد البيانات
2. إعداد قاعدة البيانات
3. Migrations
4. Schema Builder
5. أنواع الأعمدة
6. المفاتيح والفهارس
7. تعديل الجداول
8. Database Seeding
9. التمارين العملية

مقدمة في قواعد البيانات

ما هي قواعد البيانات؟

نتعامل مع قواعد البيانات بطريقة سهلة وآمنة، Laravel قاعدة البيانات هي مكان منظم لتخزين واسترجاع البيانات. في

قواعد البيانات المدعومة

يدعم 4 أنظمة قواعد بيانات Laravel:

الاستخدام	Driver	النظام
الأكثر شيوعاً	mysql	MySQL
قوي ومتقدم	pgsql	PostgreSQL
خفيف للتطوير	sqlite	SQLite
Microsoft	sqlsrv	SQL Server

إعداد قاعدة البيانات

1. إعدادات الاتصال

ملف `.env`:

```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=laravel_db
DB_USERNAME=root
DB_PASSWORD=
```

2. إنشاء قاعدة البيانات

```
-- في MySQL
CREATE DATABASE laravel_db CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
```

3. (التطوير) SQLite استخدام

```
# إنشاء ملف database
touch database/database.sqlite
```

في `.env`:

```
DB_CONNECTION=sqlite
# احذف باقي DB_* settings
```

4. اختبار الاتصال

```
php artisan migrate:status
```

❗ إذا ظهرت الرسالة بدون أخطاء، الاتصال ناجح

Migrations

Migrations ما هي

Migrations (Database للـ Version Control) هي نظام التحكم في إصدارات قاعدة البيانات

Migrations لماذا

تتبع التغييرات - معرفة من غير ماذا ومتى ؟ مشاركة سهلة - نفس البنية لجميع المطورين ؟ التراجع - إمكانية التراجع عن التغييرات ؟ نشر سهل - تطبيق التغييرات تلقائياً على الإنتاج

Migrations vs Migrations بدون

Migrations بدون ؟

- phpMyAdmin إنشاء جداول يدوياً في
- مع الفريق SQL مشاركة ملف
- تتبع التغييرات صعب
- أخطاء كثيرة عند النشر

Migrations مع ؟

- إنشاء جداول بالكود
- للمشاركة git push/pull
- تتبع تلقائي للتغييرات
- نشر سهل وآمن

Migration إنشاء

1. Migration بسيط

```
php artisan make:migration create_products_table
```

النتيجة:

```
database/migrations/2024_01_15_120000_create_products_table.php
```

2. Migration مع Model

```
php artisan make:model Product -m
```

معاً Model + Migration ينشئ.

3. هيكـل Migration

```
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**
     * Run the migrations.
     */
    public function up(): void
    {
        Schema::create('products', function (Blueprint $table) {
            $table->id();
            $table->string('name');
            $table->decimal('price', 8, 2);
            $table->integer('stock');
            $table->timestamps();
        });
    }

    /**
     * Reverse the migrations.
     */
    public function down(): void
    {
        Schema::dropIfExists('products');
    }
};
```

تشغيل Migrations

```
# تشغيل جميع Migrations
php artisan migrate

# التراجع عن آخر مجموعة
php artisan migrate:rollback

# التراجع عن جميع Migrations
php artisan migrate:reset

# إعادة تشغيل جميع Migrations
```

```
php artisan migrate:refresh

# إعادة تشغيل مع Seeding
php artisan migrate:refresh --seed

# عرض حالة Migrations
php artisan migrate:status
```

Schema Builder

إنشاء جدول

```
Schema::create('users', function (Blueprint $table) {
    $table->id(); // PRIMARY KEY AUTO_INCREMENT
    $table->string('name'); // VARCHAR(255)
    $table->string('email')->unique(); // UNIQUE
    $table->timestamp('email_verified_at')->nullable();
    $table->string('password');
    $table->rememberToken();
    $table->timestamps(); // created_at, updated_at
});
```

التحقق من وجود جدول

```
if (Schema::hasTable('users')) {
    // الجدول موجود
}

if (Schema::hasColumn('users', 'email')) {
    // العمود موجود
}
```

إعادة تسمية جدول

```
Schema::rename('old_table', 'new_table');
```

حذف جدول

```
Schema::drop('users');
```

```
Schema::dropIfExists('users');
```

أنواع الأعمدة

1. أعمدة الأرقام

```
$table->id(); // BIGINT UNSIGNED AUTO_INCREMENT
$table->bigIncrements('id'); // مثل id()
$table->integer('votes'); // INTEGER
$table->tinyInteger('status'); // TINYINT (-128 to 127)
$table->smallInteger('age'); // SMALLINT
$table->mediumInteger('quantity'); // MEDIUMINT
$table->bigInteger('views'); // BIGINT
$table->unsignedInteger('votes'); // INTEGER UNSIGNED (0+)
$table->unsignedBigInteger('user_id'); // BIGINT UNSIGNED
$table->decimal('price', 8, 2); // DECIMAL(8,2) - 999999.99
$table->float('amount', 8, 2); // FLOAT
$table->double('total', 8, 2); // DOUBLE
```

2. أعمدة النصوص

```
$table->char('code', 10); // CHAR(10) - طول ثابت
$table->string('name'); // VARCHAR(255)
$table->string('email', 100); // VARCHAR(100)
$table->text('description'); // TEXT
$table->mediumText('content'); // MEDIUMTEXT
$table->longText('article'); // LONGTEXT
$table->json('options'); // JSON
$table->jsonb('metadata'); // JSONB (PostgreSQL)
```

3. أعمدة التاريخ والوقت

```
$table->date('birth_date'); // DATE
$table->dateTime('created_at'); // DATETIME
$table->time('alarm_time'); // TIME
$table->timestamp('verified_at'); // TIMESTAMP
$table->timestamps(); // created_at + updated_at
$table->timestampsTz(); // مع timezone
$table->softDeletes(); // deleted_at (للحذف الناعم)
```

```
$table->year('graduation_year'); // YEAR
```

4. أعمدة خاصة

```
$table->boolean('is_active'); // BOOLEAN (0 or 1)
$table->enum('status', ['pending', 'active', 'blocked']);
$table->set('roles', ['admin', 'user', 'editor']);
$table->binary('photo'); // BLOB
$table->uuid('id'); // UUID
$table->ipAddress('visitor'); // IP address
$table->macAddress('device'); // MAC address
$table->geometry('positions'); // GEOMETRY
$table->point('location'); // POINT
$table->rememberToken(); // remember_token VARCHAR(100)
```

المعدلات (Modifiers)

معدلات شائعة

```
$table->string('email')->nullable(); // NULL يسمح بـ
$table->string('name')->default('Guest'); // قيمة افتراضية
$table->integer('votes')->unsigned(); // UNSIGNED (0+)
$table->string('username')->unique(); // UNIQUE
$table->decimal('price', 8, 2)->default(0); // قيمة افتراضية
$table->text('bio')->nullable(); // NULL مسموح
$table->timestamp('created_at')->useCurrent(); // وقت الإنشاء الحالي
$table->timestamp('updated_at')->useCurrentOnUpdate(); // تحديث تلقائي
```

// دمج معدلات

```
$table->string('email')->unique()->nullable();
$table->decimal('price', 8, 2)->unsigned()->default(0);
```

ترتيب الأعمدة

```
$table->string('email')->after('name'); // name بعد عمود
$table->string('name')->first(); // في البداية
```

تعليقات

```
$table->string('name')->comment('الاسم المستخدم');
```

المفاتيح والفهارس

1. Primary Key

```
$table->id(); // PRIMARY KEY تلقائي  
  
// أو يدوياً  
$table->bigInteger('id');  
$table->primary('id');
```

2. Foreign Keys (المفاتيح الأجنبية)

```
// الطريقة الحديثة (Laravel 7+)  
$table->foreignId('user_id')  
    ->constrained()  
    ->onDelete('cascade');
```

```
// الطريقة التقليدية  
$table->unsignedBigInteger('user_id');  
$table->foreign('user_id')  
    ->references('id')  
    ->on('users')  
    ->onDelete('cascade')  
    ->onUpdate('cascade');
```

```
// تسمية مخصصة  
$table->foreignId('author_id')  
    ->constrained('users') // يشير لجدول users  
    ->onDelete('set null');
```

onDelete و onUpdate خيارات

```
->onDelete('cascade') // حذف التابع عند حذف الأصل  
->onDelete('set null') // NULL تعيين  
->onDelete('restrict') // منع الحذف
```



```
->onDelete('no action') // لا شيء  
  
->onUpdate('cascade') // تحديث التابع عند تحديث الأصل
```

3. Indexes (الفهارس)

```
$table->string('email')->unique(); // UNIQUE index  
$table->index('email'); // فهرس عادي  
$table->index(['first_name', 'last_name']); // فهرس مركب  
$table->fullText('description'); // فهرس نص كامل  
  
// إزالة فهرس  
$table->dropIndex('users_email_index');  
$table->dropUnique('users_email_unique');
```

تعديل الجداول

1. إضافة أعمدة

```
php artisan make:migration add_phone_to_users_table
```

```
public function up(): void  
{  
    Schema::table('users', function (Blueprint $table) {  
        $table->string('phone')->nullable()->after('email');  
        $table->string('address')->nullable();  
    });  
}  
  
public function down(): void  
{  
    Schema::table('users', function (Blueprint $table) {  
        $table->dropColumn(['phone', 'address']);  
    });  
}
```

2. تعديل أعمدة

أولاً doctrine/dbal تثبيت

```
composer require doctrine/dbal
```

```
public function up(): void
{
    Schema::table('users', function (Blueprint $table) {
        // تغيير نوع العمود
        $table->string('name', 100)->change();

        // جعل العمود nullable
        $table->string('email')->nullable()->change();

        // تعديل عدة خصائص
        $table->decimal('price', 10, 2)->unsigned()->default(0)->change();
    });
}
```

3. إعادة تسمية أعمدة

```
Schema::table('users', function (Blueprint $table) {
    $table->renameColumn('old_name', 'new_name');
});
```

4. حذف أعمدة

```
Schema::table('users', function (Blueprint $table) {
    $table->dropColumn('phone');

    // حذف عدة أعمدة
    $table->dropColumn(['phone', 'address']);

    // أولاً ثم العمود Foreign Key حذف
    $table->dropForeign('posts_user_id_foreign');
    $table->dropColumn('user_id');
});
```

Database Seeding

Seeding؟ ما هو

Seeding = ملء قاعدة البيانات ببيانات تجريبية للتطوير والاختبار.

1. إنشاء Seeder

```
php artisan make:seeder UserSeeder
```

ملف `database/seeder/UserSeeder.php`:

```
<?php

namespace Database\Seeders;

use Illuminate\Database\Seeder;
use App\Models\User;
use Illuminate\Support\Facades\Hash;

class UserSeeder extends Seeder
{
    public function run(): void
    {
        // طريقة 1: إنشاء سجل واحد
        User::create([
            'name' => 'Admin',
            'email' => 'admin@example.com',
            'password' => Hash::make('password'),
        ]);

        // طريقة 2: إنشاء عدة سجلات
        $users = [
            [
                'name' => 'أحمد محمد',
                'email' => 'ahmed@example.com',
                'password' => Hash::make('password'),
            ],
            [
                'name' => 'سارة علي',
                'email' => 'sara@example.com',
                'password' => Hash::make('password'),
            ],
        ];

        foreach ($users as $user) {
            User::create($user);
        }
    }
}
```

```

    }

    // طريقة 3: استخدام Factory
    User::factory()->count(50)->create();
}
}

```

2. تشغيل Seeder

في `database/seeder/DatabaseSeeder.php`:

```

public function run(): void
{
    $this->call([
        UserSeeder::class,
        ProductSeeder::class,
        CategorySeeder::class,
    ]);
}

```

تشغيل:

```

# تشغيل Seeders جميع
php artisan db:seed

# تشغيل seeder محدد
php artisan db:seed --class=UserSeeder

# seeding مع migrations إعادة تشغيل
php artisan migrate:fresh --seed

```

3. Model Factories

```

php artisan make:factory ProductFactory

```

ملف `database/factories/ProductFactory.php`:

```

<?php

namespace Database\Factories;

use Illuminate\Database\Eloquent\Factories\Factory;

```

```
class ProductFactory extends Factory
{
    public function definition(): array
    {
        return [
            'name' => fake()->words(3, true),
            'description' => fake()->paragraph(),
            'price' => fake()->randomFloat(2, 10, 1000),
            'stock' => fake()->numberBetween(0, 100),
            'is_active' => fake()->boolean(80), // 80% true
        ];
    }
}
```

الاستخدام:

```
// إنشاء منتج واحد
$product = Product::factory()->create();

// إنشاء 50 منتج
Product::factory()->count(50)->create();

// مع خصائص محددة
Product::factory()->create([
    'name' => 'لايتوب',
    'price' => 5000,
]);
```

أمثلة عملية

مثال 1: جدول المنتجات

```
Schema::create('products', function (Blueprint $table) {
    $table->id();
    $table->string('name');
    $table->string('slug')->unique();
    $table->text('description')->nullable();
    $table->decimal('price', 10, 2);
    $table->integer('stock')->default(0);
    $table->boolean('is_active')->default(true);
    $table->foreignId('category_id')
        ->constrained()
```

```

        ->onDelete('cascade');
    $table->timestamps();
    $table->softDeletes();

    // فهرس
    $table->index('is_active');
    $table->index('created_at');
});

```

مثال 2: جدول الطلبات

```

Schema::create('orders', function (Blueprint $table) {
    $table->id();
    $table->string('order_number')->unique();
    $table->foreignId('user_id')->constrained()->onDelete('cascade');
    $table->decimal('total_amount', 10, 2);
    $table->enum('status', ['pending', 'processing', 'completed', 'cancelled'])
        ->default('pending');
    $table->timestamp('paid_at')->nullable();
    $table->timestamps();
});

```

(Many-to-Many) مثال 3: جدول محوري

```

Schema::create('post_tag', function (Blueprint $table) {
    $table->id();
    $table->foreignId('post_id')->constrained()->onDelete('cascade');
    $table->foreignId('tag_id')->constrained()->onDelete('cascade');
    $table->timestamps();

    // فهرس مركب فريد
    $table->unique(['post_id', 'tag_id']);
});

```

أفضل الممارسات

أفعل ؟

1. استخدم أسماء واضحة

```
// جيد
Schema::create('user_profiles', ...);
Schema::create('product_categories', ...);
```

```
// سيء Schema::create('up', ...); Schema::create('pc', ...);
```

```
2. **فقط عند الحاجة nullable اجعل الأعمدة**
```php
// جيد
$table->string('email'); // مطلوب
$table->string('phone')->nullable(); // اختياري

// سيء - كل شيء nullable
$table->string('name')->nullable();
$table->string('email')->nullable();
```

### 3. Foreign Keys استخدم

```
// جيد
$table->foreignId('user_id')->constrained()->onDelete('cascade');
```

```
// سيء $table->unsignedBigInteger('user_id');
```

```
4. **للبحث المتكرر Indexes أضف**
```php
$table->index('email');
$table->index('status');
$table->index(['user_id', 'created_at']);
```

5. Enum للقيم المحدودة استخدم

```
$table->enum('status', ['active', 'inactive', 'pending']);
$table->enum('role', ['admin', 'user', 'moderator']);
```

لا تفعل

1. القديمة migrations لا تعدّل

```
// قديم migration سيء - تعديل
```

```
جديد بدلاً من ذلك migration // أنشئ
```

```
// جيد ❹ php artisan make:migration add_phone_to_users_table
```

```
2. **في الإنتاج migrations لا تحذف**  
```php  
// خطر جداً ❹
php artisan migrate:fresh // في الإنتاج!

// آمن ❹
php artisan migrate
```

### 3. **down() method** لا تنسَ

```
// سيء ❹
public function down(): void
{
 // فارغ
}
```

```
// جيد ❹ public function down(): void { Schema::dropIfExists('products'); }
```

```

التمارين العملية ##

Categories ❶ ### تمرين 1: إنشاء جدول

```bash  
php artisan make:migration create_categories_table
```

```
public function up(): void  
{  
    Schema::create('categories', function (Blueprint $table) {  
        $table->id();  
        $table->string('name');  
        $table->string('slug')->unique();  
        $table->text('description')->nullable();  
        $table->boolean('is_active')->default(true);  
        $table->timestamps();  
    });  
}
```



```
}
```

Foreign Key مع Posts تمرين 2: جدول

```
Schema::create('posts', function (Blueprint $table) {
    $table->id();
    $table->string('title');
    $table->string('slug')->unique();
    $table->text('content');
    $table->foreignId('user_id')->constrained()->onDelete('cascade');
    $table->foreignId('category_id')->constrained()->onDelete('set null');
    $table->integer('views')->default(0);
    $table->boolean('is_published')->default(false);
    $table->timestamp('published_at')->nullable();
    $table->timestamps();
    $table->softDeletes();
});
```

البيانات Seeder: تمرين 3

```
class CategorySeeder extends Seeder
{
    public function run(): void
    {
        $categories = [
            ['name' => 'تقنية', 'slug' => 'tech'],
            ['name' => 'رياضة', 'slug' => 'sports'],
            ['name' => 'أخبار', 'slug' => 'news'],
        ];

        foreach ($categories as $category) {
            Category::create($category);
        }
    }
}
```

ملخص ^٤

في هذا الدرس، تعلمت

جميع أنواع الأعمدة المتاحة ^٤ المفاتيح الأجنبية والفهارس ^٥ Migrations إعداد قاعدة البيانات والاتصال ^٤ إنشاء وتشغيل ^٤

موارد إضافية [؟](#)

- [Laravel Migrations Documentation](#)
 - [Database Seeding](#)
 - [Schema Builder](#)
-

اختبر نفسك [؟](#)

1. `migrate` و `migrate:fresh` ما الفرق بين
 2. على العمود؟ `nullable()` متى نستخدم
 3. Foreign Key كيف تنشئ
 4. `onDelete('cascade')` ما فائدة
 5. Seeder و Factory ما الفرق بين
-

الدرس التالي

الأساسيات - Eloquent ORM : جاهز للمزيد؟ انتقل إلى **الدرس 6**

في الدرس 6، ستتعلم

- Eloquent ORM ما هو
 - Models إنشاء والاستعلام عن
 - العلاقات بين الجداول
 - والمزيد!
-